

Modules, Macros, and Build Systems

Document: [P1052R0](#)
Audience: EWG, SG15
Authors: Boris Kolpackov
Reply-To: boris@codesynthesis.com
Date: 2018-05-02

Contents

- 1 [Abstract](#)
- 2 [Background](#)
- 3 [Modules with Macros](#)
- 4 [The Atom Proposal](#)
- 5 [The Merged Proposal](#)
- 6 [Acknowledgements](#)

1 Abstract

One of the main challenges of building modularized projects is discovering the set of modules imported by each translation unit. This is a relatively straightforward process if modules are a purely language-level mechanism (as is currently the case in Modules TS). If, however, modules start affecting the preprocessor (for example, by supporting exportation of macros), then we believe this discovery will result in

complexity that most build system (and other tools) vendors will have no capacity to handle.

While it appears that the authors of the Atom proposal have recognized this issue and tried to resolve it, we believe their current approach is unworkable. While there appears to be no immediate plans to support exportation of macros either in IS or Modules TS, there are plans to merge legacy header modules from the Atom proposal into Modules TS which would make the combined result still suffering from these problems.

2 Background

In order to build a project that uses modules, a build system needs to obtain *module dependency information* – a set of modules imported by each translation unit (TU). This information is necessary both to establish the order in which TUs can be compiled as well as which TUs must be recompiled. Specifically, a module interface unit must be compiled (into a binary module interface or BMI) before any TU that imports it and if a module interface unit has changed, then all the TUs that import it must be recompiled.

Just like with header dependency information, we believe having the user specify module dependency information manually is not a scalable approach.

Compiling all the module interface units as an *ad hoc* pre-build step is not a workable approach since module interface units may import each other.

Extracting module dependency information in Modules TS ([N4720](#)) is a relatively straightforward process: the TU is preprocessed, tokenized, and *shallow-parsed* to collect the names of imported modules. The parsing can be shallow because all the module-related declarations are *top-level* and the parser can safely ignore all the tokens inside `{ }`. [Note: With the exception for `export { import M; }` which is still easy to recognize and handle.]

Extracting module dependency information can naturally be combined with the header dependency extraction which requires essentially a full preprocessor run.

While we would expect the compiler vendors to provide this functionality (perhaps combined with the header dependency extraction that they already provide), our experience with `build2` shows that this can also be implemented by the build system (or other tools) with good results (simple and reliable code with good performance).

3 Modules with Macros

Making modules capable of exporting macros will make `import` a preprocessor directive rather than (or, more precisely, in addition to) a language declaration since via exported macros it will now be able to affect the preprocessor state. This "hoisting" of `import` into the preprocessor will significantly complicate module dependency extraction.

Specifically, the approach described in the previous section will no longer work since a previously-imported module may now effect (via a preprocessor macro) the importation of subsequent modules. For example:

```
import foo; // May export macro F00.

#ifdef F00
import bar;
#endif
```

In this model the build system will no longer be able to determine the module dependency information at the outset, before starting the compilation. And the compiler may not have access to all the (up-to-date) BMIs to perform the compilation. As a result, the compiler will have to *call back* into the build system on encountering every `import` directive in order to obtain an (up-to-date) BMI that it can use (and which the build system might still have to compile, potentially triggering a recursive chain of callbacks).

Besides the sheer complexity of this approach, implementing it in a parallel build system immediately presents a number of practical challenges (contemplating challenges for a distributed build system is left as an exercise for the reader):

1. In this model discovery of imported modules is inherently a serial operation. Which means, for a single TU, building of its imported modules is non-parallelizable. While this may not be an issue for most from-scratch builds (since we will, presumably, be compiling multiple TU that import different sets of modules and/or in different order), this can become a major problem for incremental builds.
2. A callback into a parallel build system may determine that the requested module interface unit is already being compiled (as a result of being imported by another TU) and therefore would have to wait. In this case, to achieve full resource utilization, the build system would have to reuse the "job" to compile another TU (which can again get blocked and be reused, recursive). As a result, in this model, we may end up with a large number of waiting compiler processes that still holdup other (than CPU) system resources (most critically, RAM).

4 The Atom Proposal

We understand that the authors of the Atom Proposal ([P0947R1](#)) have recognized this issue and tried to resolve it by imposing a number of restrictions on the location of the import directives/declarations as well as the kind of macro expansions that can be performed around them. Specifically, [Section 4.3, "Preprocessor Impact"](#) states:

"We also wish to permit the set of imports of a translation unit to be determined without knowledge of the contents of the imported translation units. In particular, the full set of dependencies should be discoverable (for instance, by a build tool or a non-compiler parser of source code) without the need to consult external files, safe in the knowledge that no macro will (for instance) `#define import`. However, preprocessor action should still be permitted in the import declaration region, to allow constructs such as:

```
#ifdef BUILDING_ON_UNIX
import support.unix;
#else
import support.windows;
#endif
```

To this end, macro expansion before the end of the initial sequence of import-declarations is disallowed from expanding an imported macro."

Firstly, we believe it will be hard for external tools to extract the imported module set without support from the compiler because the required semantics will have to be along these lines:

Preprocess the import region and stop (we have to stop since continuing preprocessing requires BMIs). Then parse the import declarations and return the set of imported module names.

The stop part is something that external tools will have a hard time doing without some sort of support from the compiler.

The second, bigger, issue is how to stop. We don't see how this can be achieved without loading the BMIs because without doing so there is no way of knowing whether a macro is module-exported or not. Consider these two side-by-side examples:

```
import foo; // May export F00. | import foo; // Doesn't export F00.
                                |
#ifdef F00                     | #ifdef F00
#  error need F00               |   import bar;
#endif                         | #endif
```

Without loading `foo`'s BMI the two cases are indistinguishable. However, the compiler has to somehow stop before `#ifdef` on the left hand side but continue on the right.

5 The Merged Proposal

From [P0983R0](#) we understand that while there are currently no plans to add support for exportation of macros into IS or Modules TS, there are plans to merge legacy header modules from the Atom proposal into Modules TS. Such modules will be able to export macros and therefore the combined result will be affected by the problems described in the previous sections.

6 Acknowledgements

Thanks to Nathan Sidwell and Jens Maurer for comments on early drafts of this paper.